Kiosk App — Developer Handover

witworldwide tourist kiosk · repo `wit-project-sku/kiosk-electron` · v5.0.27 · updated 2026-10-21

1. What it is
2. Run it on your PC
3. Folder structure
4. How the app is wired
5. Locations and kioskId
6. Data: where it comes from, when, where it's saved
7. Google Sheets in detail
8. Files on a kiosk machine
9. Daily power schedule (auto shutdown)
10. Releases and auto-update
11. .env explained
12. Common tasks
13. Gotchas

1. What it is

A touch-screen tourist information kiosk (Electron 34 + React 19 + TypeScript, CSS Modules). Windows only. **One build runs at every location.** Each machine is told which location it is (its **kioskId**, e.g. `W004`), and that picks the screens, the content and the features (card terminal, camera, games…).

Main features: info screens (shops, events, palaces, transport, tax-free, exchange rate, weather, flights/ferries in Jeju), AI course recommendation, AR hanbok photo booth with QR download, donation/shop web pages with card payment, footfall counting, Jeju motion games, height measurement (Jeju, ZED camera).

2. Run it on your PC

1. Install Node 20+. Run `npm install` (it also rebuilds native modules for Electron).
2. Get the real `.env` and `secrets/service-account.json` + `secrets/service-account-osaek.json` from the previous developer (never in git). `.env.example` shows every key.
3. Set `DEV_MODE=true` in `.env` → a location switcher appears on the home screen so you can view any kiosk (W001…W202).
4. `npm run dev` → app with hot reload. `F12` opens DevTools.

Command	What it does
`npm run dev`	Run in development
`npm run typecheck` / `npm run lint`	Check before committing
`npm run sync:sheet`	Pull Google Sheets → regenerate `src/renderer/src/data/*.generated.ts` (see §7)
`npm run build:win`	Production installer (bumps patch version)
`npm run build:win:beta`	Beta installer (installs side by side with production)

3. Folder structure

```
kiosk-app/
├─ src/
│  ├─ main/                ← BACKEND (Electron main process, Node). All logic, network, DB.
│  │  ├─ index.ts          startup order: DB → services → power → windows → sync/updates
│  │  ├─ container.ts      creates every service (one place to see what exists)
│  │  ├─ core/             app-wide basics
│  │  │    KioskConfigStore.ts   reads/writes kioskId  (kiosk-config.json)
│  │  │    AppConfigStore.ts     app settings          (settings.json)
│  │  │    PowerManager.ts       daily reboot / shutdown / auto-start
│  │  │    env.ts                loads .env            paths.ts  all disk paths
│  │  │    PaymentAgentManager.ts starts the card-payment agent
│  │  │    ZedSidecarManager.ts   starts the Jeju height camera (Python)
│  │  │    logger.ts             log files
│  │  ├─ services/         one service per feature: ShopService, BannerService, WeatherService,
│  │  │    ExchangeService, FlightService, SailingService, OutfitService, StatsService …
│  │  │    sync/            night sync + Google Sheets (SyncService, GoogleSheetsSyncTransport)
│  │  │    photo/ ai/ camera/  AR hanbok photo flow       footfall/  people counting
│  │  │    height/ motion/ spotDiff/ drive/  Jeju extras, games, Google Drive upload
│  │  ├─ database/         SQLite: Database.ts, migrations/ (001…008), repositories/
│  │  ├─ ipc/handlers/     the API the UI calls (one file per feature: shop.handlers.ts …)
│  │  ├─ updater/          auto-update + remote "update now" command
│  │  └─ windows/          main window + customer display window
│  ├─ preload/             safe bridge: exposes main's IPC API to the UI as window.api
│  ├─ renderer/src/        ← FRONTEND shell (React)
│  │    App.tsx            picks the layout for this kioskId
│  │    store/             Zustand stores (shopStore, weatherStore, languageStore …)
│  │    hooks/             useKioskController (navigation + idle reset), usePhotoWorkflow …
│  │    lib/               small helpers (i18n, shops, videoMap …)
│  │    data/              *.generated.ts from Google Sheets + sheet-cache/ (raw CSV)
│  ├─ layouts/             ← THE SCREENS. One folder per location family
│  │    InsadongLayout.tsx  insadong/   (W001, W002; W003 = NamInsadongLayout)
│  │    OsanLayout.tsx      osan/       (W004 오색시장)
│  │    HwaseongLayout.tsx  hwaseong/   (W005 화성휴게소)
│  │    JejuLayout.tsx      jeju/       (W006 공항, W007 여객터미널, W008 세계자연유산)
│  │    KadaLayout.tsx      kada/       (W202 Vietnam)
│  │    photo/              shared AR photo screens     components/  shared widgets
│  └─ shared/              used by BOTH sides
│       config/kioskLocations.ts   ★ list of all locations and their features
│       types/  constants/  ipc/   (IPC channel names + contracts)
├─ themes/                 colours/fonts per location (JSON)
├─ resources/              mediapipe (camera AI models), videos (dev only), help images
├─ payment-agent/          compiled card-payment server (separate repo; don't edit here)
├─ zed-height/             Python script for the ZED height camera (Jeju)
├─ scripts/                build, sync-sheet, provisioning, self-tests, db-inspect
├─ tools/location-tester/  switch an installed kiosk to another location
├─ secrets/                Google service-account JSON (gitignored)
├─ docs/                   AUTO_UPDATE.md, FOOTFALL.md, UPDATE_COMMAND_API.md, this file
├─ provision-kiosk.ps1     set a machine's kioskId
├─ Set/UnsetAutoShutdown*.bat  OLD manual power scripts (app now does this itself, §9)
└─ electron-builder.yml    installer config (.beta.yml for beta)
```

Rule of thumb: data, network, files, timers → `src/main/services`. What the visitor sees → `src/layouts/<location>`. The UI never calls the internet directly; it asks main through `window.api` (IPC).

4. How the app is wired

```
Google Sheets / witteria API / weather / flights
          │  (fetched by)
          ▼
  src/main/services/*  ──save──▶  SQLite  (%APPDATA%\kiosk-app\data\kiosk.db)
          │
  src/main/ipc/handlers/*   (main answers requests, pushes "changed" events)
          │
  src/preload  → window.api
          │
  renderer stores (Zustand)  →  src/layouts/<location>/*.tsx  (screens)
```

- **Startup** (`src/main/index.ts`): load .env → open DB + migrations → create services → set power schedule → open window (shows cached data instantly) → start weather/flights/exchange polling → start payment agent (if the location has a card terminal) → start auto-update → refresh content in background.
- **Screens**: every layout is a fixed **2160 × 3840** canvas scaled to the monitor, so Figma px = CSS px.
- **Navigation and idle**: `useKioskController.ts`. After 3 minutes without touch the kiosk goes back to home and resets language to Korean.
- **Languages**: ko / en / ja / zh. Text comes from the Localization tab of the location's Google Sheet.

5. Locations and kioskId

kioskId	Place	Layout	Card terminal
W001	북인사마당	INSADONG	no
W002	인사동쉼터	INSADONG	no
W003	남인사마당	NAM_INSADONG	yes
W004	오산 오색시장	OSAN	yes
W005	화성휴게소	HWASEONG	yes
W006	제주공항	JEJU_AIRPORT	yes
W007	제주국제여객터미널	JEJU_AIRPORT	yes
W008	세계자연유산본부	JEJU_HERITAGE	yes
W202	KADA (Hà Nội, Vietnam)	KADA	no

Source of truth: `src/shared/config/kioskLocations.ts` (features per location: card terminal, donation, coordinates, camera rotation…).

Where kioskId is saved

- File: `%APPDATA%\kiosk-app\kiosk-config.json` → `{"kioskId":"W004"}` (beta: `%APPDATA%\kiosk-app-beta\`).
- Read by `src/main/core/KioskConfigStore.ts`. It is **not** in .env and **not** in the build, so one installer fits every location.
- Set it on a new machine (run once, then restart the app):

  ```
  powershell -ExecutionPolicy Bypass -File provision-kiosk.ps1 W004
  ```

The script is also copied next to the installed app. Optional `-ShopId N` overrides the shop-API id (normally leave it off). `-Beta` for the beta install.

- Not provisioned → falls back to the default kioskId in `src/shared/constants`.

6. Data: where it comes from, when, where it's saved

Data	Source	When fetched	Saved in
UI text / translations	Google Sheet (Localization tab)	On every launch + 00:00 nightly	SQLite `translations`; fallback = `*.generated.ts` in the build
Palaces, AI categories, markets, Jeju facilities	Google Sheet	Build time only (`npm run sync:sheet`)	`src/renderer/src/data/*.generated.ts`
Shops, attractions, banners, outfits, AR backgrounds, game rounds	witteria API	On launch + 00:00 nightly	SQLite `local_cache`; images in `remote-images/`
Home button layout	witteria API	On launch only	SQLite
Video subtitles	witteria API (Jeju: sheet fallback)	On launch	SQLite
Weather	OpenWeatherMap	Every 30 min	memory
Exchange rates	Korea Eximbank API	Every 6 h	memory/cache
Flights / ferries (Jeju)	data.go.kr / KOMSA	Every 2–10 min	memory/cache
Touch stats, photo shots	→ witteria API	Sent live; failures retried at 00:00	SQLite `analytics_events`, `failed_requests`
Footfall counts	camera → FOOTFALL_API_URL	Uploaded 21:30	SQLite `footfall_buckets`
Photos taken	camera + AI server	—	`C:\KioskPhotos`

If the internet is down the kiosk keeps working on the last saved data. Nothing on screen waits for the network.

7. Google Sheets in detail

Each location family has **its own spreadsheet**: Insadong (W001–W003), Osaek (W004), Hwaseong (W005), Jeju (W006–W008). KADA has none (its text is in `layouts/kada/kadaText.ts`). The sheet IDs are in `GoogleSheetsSyncTransport.ts` (runtime) and `scripts/sync-sheet.mjs` (build time) — keep both in sync.

Two ways sheets are read

1. **Build time** — `npm run sync:sheet` downloads tabs like `Localization_Insa`, `PalaceInfo_Insa`, `AICategory_*` and writes typed files to `src/renderer/src/data/` (raw CSV cached in `sheet-cache/`). These ship inside the app as the offline default. Run it before a release when content changed.
2. **Runtime** — `SyncService` + `GoogleSheetsSyncTransport` read the location's `Localization_*` tab **on every app launch** and **every night at 00:00** (`NightSyncScheduler.ts`), and save it to SQLite (`translations` table). The new text shows after the next reload.

How it authenticates

- A Google **service account** (`secrets/service-account*.json`, path in `GOOGLE_SERVICE_ACCOUNT_JSON`). Each spreadsheet must be shared with that service account's email.
- `GOOGLE_SHEETS_ID` is only an **on/off switch** — any value turns night sync on. Which sheet is read depends on kioskId.

Nightly job at 00:00 does

refresh shops/attractions/banners/outfits/backgrounds/games → resend failed stats → download sheet text → upload analytics → retry Drive photo uploads → prune old height rows.

8. Files on a kiosk machine

What	Where
Installed app (+ bundled .env, secrets, themes, provision-kiosk.ps1)	Install dir → `resources\`
kioskId	`%APPDATA%\kiosk-app\kiosk-config.json`
Settings	`%APPDATA%\kiosk-app\settings.json`
Database (all cached content, stats, queues)	`%APPDATA%\kiosk-app\data\kiosk.db`
Logs (10 MB rotation) — look here first when something breaks	`%APPDATA%\kiosk-app\logs\`
Downloaded CMS images	`%APPDATA%\kiosk-app\remote-images\`
Photos (captured + AI)	`C:\KioskPhotos` (or `PHOTO_SAVE_DIR`)
Attract / display videos (copied by hand, survive updates)	`C:\KioskVideos\<set>\` (or `KIOSK_VIDEOS_DIR`)

Beta build uses `%APPDATA%\kiosk-app-beta\` for everything, so both can be installed on the same PC.

9. Daily power schedule (auto shutdown)

The app sets this up **by itself on every launch** (`src/main/core/PowerManager.ts`) using Windows Scheduled Tasks, and registers itself to auto-start on login.

Who	Schedule	Windows task name
W001, W002, W003, W005, W202	Reboot every day at **02:00**	`KioskAutoRestart`
W004, W006, W007, W008	**Shut down 22:00**, start **08:00**	`KioskShutdownAt10PM`, `KioskStartAt8AM`
Beta build	Reboot every day at **09:00**	`KioskBetaAutoRestart`

> **Important:** Windows cannot turn on a PC that is fully powered off. For the 22:00/08:00 sites, set the motherboard **BIOS** → **"Power On by RTC Alarm" = 08:00**, otherwise the kiosk stays off in the morning.

The `SetAutoShutdown*.bat` / `UnsetAutoShutdown*.bat` files in the repo root are the old manual way — not needed any more. To check on a machine: open *Task Scheduler* and look for the task names above. The Jeju hours (22:00/08:00) are copied from 오색시장 and should be confirmed with each venue.

10. Releases and auto-update

- `npm run build:win` builds the installer and publishes to **GitHub Releases**. Kiosks update themselves (electron-updater).
- **Production** kiosks check once a week (`UPDATE_DAY` / `UPDATE_TIME`, default Friday 17:00; a missed window is caught up at next start). **Beta** kiosks check every 15 min and only take beta releases.
- Install happens only when the kiosk is idle (never during photo countdown / AI generation).
- **"Update now"**: an admin button on the witteria site; kiosks poll it every 5 min and update immediately. See `docs/UPDATE_COMMAND_API.md`.
- The `.env` and `secrets/` on your PC are **baked into the installer** — build from a machine with the correct production `.env`.
- More: `docs/AUTO_UPDATE.md`.

11. .env explained

Loaded by `src/main/core/env.ts`: in dev from the project root, in the installed app from `resources\.env`. Changing it on a kiosk needs only an app restart. **Never commit it.**

Key	Meaning	Needed?
External APIs		
`OPENWEATHER_API_KEY`	Weather card + Jeju forecast	yes
`EXIM_API_KEY`	Exchange rates (Korea Eximbank)	yes
`DATA_GO_KR_SERVICE_KEY`	Public data portal (Jeju flights)	Jeju
`KOMSA_SERVICE_KEY`	Ferry schedules (Jeju port)	Jeju
`WITTERIA_API_BASE`	Main backend: shops, banners, events, stats, photo QR upload. Use the PROD host.	yes
`VITE_AR_PROCESS_IMAGE_API_URL`, `VITE_AR_PROCESS_API_URL`	AI hanbok photo servers (solo / together)	photo booth
Google Sheets		
`GOOGLE_SHEETS_ID`	On/off switch for the sheet sync (value itself not used)	yes
`GOOGLE_SERVICE_ACCOUNT_JSON`	Path to the service-account file, e.g. `secrets/service-account.json`	yes
Card payment (only W003–W008, which have a terminal)		
`PAYMENT_HTTP_HOST` / `_PORT`	Local payment server address (`127.0.0.1:8080`)	yes
`PAYMENT_CORS_ORIGINS`	Web pages allowed to call it: the witteria store AND the Vercel donation app	yes
`PAYMENT_CENTRAL_BASE_URL`, `_AUTH_TOKEN`, `_TIMEOUT_MS`	Central payment server	yes
`PAYMENT_TL3800_*`	Card terminal (TL-3800): terminal ID, serial port settings (blank port = auto-detect), retry timings	yes
`PAYMENT_OUTBOX_*`	Retry queue for payments that failed to reach the server	defaults ok

`PAYMENT_LOG_LEVEL`	Log detail of the payment agent	defaults ok
Updates		
`UPDATE_CHANNEL`	`latest` = production, `beta` = test kiosks	yes
`UPDATE_DAY`, `UPDATE_TIME`	Weekly update check (default Friday 17:00)	optional
`UPDATE_BETA_INTERVAL_MIN`	Beta check interval (default 15)	optional
`UPDATE_COMMAND_POLL_MIN`, `UPDATE_COMMAND_API_URL`	"Update now" polling	optional
Footfall (people counting)		
`FOOTFALL_ENABLED`, `_LINE`, `_FPS`, `_SCORE`, `_TRACK_SCORE`, `_MIN_BOX`	Tuning; defaults are what the fleet uses. See `docs/FOOTFALL.md`	optional
`FOOTFALL_API_URL`, `FOOTFALL_API_KEY`	Where to upload counts at 21:30. Empty = keep locally.	optional
Dev / machine		
`DEV_MODE`	`true` shows the location switcher. **Must be false on live kiosks.**	dev
`KIOSK_VIDEOS_DIR`	Folder of attract videos (dev: your local copy)	dev
`PHOTO_SAVE_DIR`	Override `C:\KioskPhotos`	optional

The kioskId is intentionally **not** an env var (see §5).

12. Common tasks

I want to…	Do this
Change a text on screen	Edit the location's Google Sheet Localization tab. Kiosks pick it up at next launch / midnight. Also run `npm run sync:sheet` before the next release.
Change a screen design	`src/layouts/<location>/` (component + `.module.css`). Colours: `themes/`.
Add a new location	Add it to `kioskLocations.ts`, pick/make a layout, add its sheet in both sheet-ID places, provision the machine.
Add data from a new API	Service in `src/main/services` → register in `container.ts` → IPC handler in `ipc/handlers` + channel in `shared/ipc` → store in `renderer/src/store` → use in layout.
Change the DB	Add a new migration file (`009_…ts`) — never edit old ones.
Install a new kiosk	Install → open once → `provision-kiosk.ps1 W00x` → copy videos to `C:\KioskVideos` → set BIOS RTC (22:00/08:00 sites) → restart.
Debug a kiosk	Read `%APPDATA%\kiosk-app\logs\`. `scripts/db-inspect.mjs` reads the DB.

13. Gotchas

- `README.md` and `PROJECT_STATE.md` are **outdated**. Trust the code and this document.
- `@nestjs/*` and `serialport` in package.json are for the bundled payment agent, not for the app.
- Some code comments say night sync is at 02:00 — it is really **00:00** (`NightSyncScheduler.ts`).

- The home screen does not reset on idle if it's already on home: any popup on the home screen must close itself (`useInactivityReset`).
- Camera AI (MediaPipe) is bundled offline in `resources/mediapipe` and served via `appres://`; refresh with `npm run vendor:mediapipe`.
- Never commit `.env`, `secrets/`, or photos.